Same prediction now, different memory later

A small GPU experiment on transformer memory and density matrices

Research and search date: 21 October 2026 (UTC). Prepared for a master's project. This is a proposed experiment, not a report of GPU results.

Recommendation. Train one small transformer to remember a noisy clue while reading irrelevant tokens. Find pairs of histories that require exactly the same next-token probabilities, then append the same query and measure their different answers. Exchange saved memory between the histories. Test whether a tiny density matrix predicts the resulting behavior better than equally small classical models.

The useful contribution would be a carefully measured test of whether a compact description remains accurate after memory interventions. Recovering predictive information, displaying belief geometry, and showing that internal memory affects predictions are already established ideas. The literature now includes especially close intervention experiments from September 2026.

Treat the density matrix as an optional modeling constraint. For the generator recommended here, one ordinary probability already describes everything relevant about the hidden bit. A density matrix might help approximate the trained transformer's particular behavior, but this task provides no reason to expect a fundamental quantum advantage.

What was actually done for this report. I searched primary papers and public repositories, read the three starting papers beyond their abstracts, inspected relevant analysis and intervention code, and checked the worked probabilities with exact rational arithmetic on a CPU. I also checked the proposed operator normalization numerically. I did not train a transformer, fit an encoder, run a GPU intervention, reproduce the papers, or benchmark runtime.

Reading order. Sections 1-6 give the contribution, workflow, generator, settings, measurements, and overlap table. Appendix A gives the mathematics. Appendix B gives pseudocode. Appendix C records source access and code inspection. References are clickable.

1. Candidate contribution and the closest work

1.1 What the literature already establishes

Riechers, Elliott, and Shai recover known predictive states from neural activations for classical, quantum, and post-quantum synthetic processes. Their states already describe distributions over complete futures. Their main analysis fits affine maps from concatenated last-position activations to known generator states; Appendix F describes probability weighting, deduplication, and cross-validation. Their comparisons include finite-order classical approximations. This is substantially more than recovering the current output distribution. It is not, however, the same test as fitting a small model of a frozen transformer's own future probabilities and testing it after partial cache exchanges. [1]

Shai and colleagues make the next-token ambiguity particularly explicit with Random-Random-XOR. Known belief information can be recoverable across layers even when the final residual vector does not reveal it through the tested linear probe. A residual vector is the internal feature vector passed between transformer blocks. Their experiments already undermine the claim that a model trained on next-token prediction must represent only next-token probabilities. Their regression and training details appear in Appendix A. [2]

Piotrowski and colleagues go beyond geometric recovery: they analyze the computations producing intermediate representations. Their constrained belief-update account predicts attention weights, value-output transformations, and embeddings in small transformers. Their appendices examine training, multiple layers, and architectural requirements. Thus, proposing to study how predictive representations update is also not automatically new. Their strongest mechanistic account has a specific small-model and generator scope. [3]

The most important subsequent overlap is Balcells and colleagues, September 2026. They study belief representations in pretrained language models, intervene along directions that preserve the generator's complete next-token distribution, and investigate prediction changes under belief patching or steering. Their prediction interventions vary the edited suffix length and measure the final position's distribution. The inspected code evaluates a modified suffix using cached prefix information. This already covers belief interventions, next-token-invisible belief directions, and competition with unedited context. It does not establish the precise autonomous, compact-state, partial-cache transfer comparison proposed below. [4]

An earlier public capstone by Balcells and García Segura also uses an encoder and decoder to inject alternative beliefs into a small Mess3 transformer and score next-token changes. It is relevant prior experimentation even though it is not a peer-reviewed paper. A learned decoder is not guaranteed to be a true inverse, and arbitrary decoded activations need not correspond to a natural history. [5]

1.2 Four different levels of evidence

Level	What must be demonstrated	What it does not establish
Recover a known predictive state	An activation probe predicts the generator's posterior or predictive coordinates on held-out histories	That the recovered coordinates explain all of the trained model's errors or cause its output
Predict model behavior	A summary predicts the frozen transformer's full probability distributions on unseen histories	That the summary updates correctly or survives interventions
Update the summary	Start from one summary; supply subsequent tokens; predict later distributions without rereading transformer activations	That changing the corresponding model information has the predicted causal effect
Test causal use	Modify a specified model variable and predict the measured change in later outputs	That the representation is unique, globally sufficient, or physically quantum

A probe is a fitted readout used to test what information can be recovered from internal numbers. A predictive state is a description sufficient for specified future predictions. A causal intervention changes part of a model's computation while holding other parts fixed. These are different tests.

Predictive-state representations already define state using predictions of future observations and give recursive updates. Hidden quantum Markov models already use density matrices with probability-producing updates. Neither the probability chain rule nor a matrix update formula would be a contribution here. [6-8]

Causal memory studies also precede this proposal. Othello-GPT edits internal board representations; cache-steering and Memory Inception alter saved attention information to change later behavior. Their existence rules out claiming the first causal use of transformer memory or the first useful cache intervention. [9-11]

A public J-state closure research repository describes compact-state prediction, persistent-cache attribution, and intervention-retention protocols. Its own documentation separates implemented protocols from successful results. It is additional reason not to claim novelty for the broad idea of testing compressed states causally. [13]

1.3 The narrow candidate

Candidate contribution: On histories matched in their current full output distributions, quantify how well a three-real-number summary, fitted only on unmodified histories, predicts the frozen transformer's conditional future distributions after exchanging earlier-token versus current-token KV memory. Compare a two-dimensional density-matrix model with a four-state classical probabilistic model and a three-number recurrent neural model, under matched information and parameter budgets.

KV memory means the keys and values saved by attention for previously processed tokens. Keys help choose which information to read; values carry the information being read. The important test is transfer from ordinary histories to interventions, including the direction and size of future changes. The density-matrix constraint is assessed through predictive errors, not appearance.

Relative to [1-3], this targets the trained model's behavior, learned autonomous updates, and partial cache interventions. Relative to [4-5], it predicts several subsequent token steps using one initial compact summary, evaluates complete short-future distributions, and explicitly separates

earlier-position from current-position persistent memory. Relative to [10-11,13], it uses an exact oracle with matched-now/different-later pairs and a controlled comparison of state constraints.

I found no study matching this entire narrow comparison in the accessible sources inspected. That is not proof of originality. The ingredients individually overlap prior work, and one potentially relevant OpenReview submission could not be accessed. This is a feasible empirical question and a defensible pilot, not a guaranteed publication claim.

Important limitation. Ordinary histories can make several memory locations carry redundant, correlated information. Fitting on ordinary histories alone cannot generally identify which location will matter after they are put into conflict. Success on cache hybrids would therefore be useful empirical transfer; failure is also plausible and informative. It would not disprove density matrices in general.

2. One numbered experimental workflow

1. **Write and verify the exact generator.** Implement the rules in Section 3 and an oracle, meaning a program that computes the correct probabilities from those rules. Verify normalization, the matched pair, reset equivalence, and termination. Use exact fractions for the small worked example.

2. **Create disjoint history families before training.** Assign all episodes sharing the same 16-token irrelevant prefix to one split. Keep both cue values, every continuation, every reset version, and every intervention donor from that family in the same split. Reserve separate families for transformer training, transformer validation, analysis fitting, analysis validation, and final testing.

3. **Run a small correctness trial.** Use one seed and 300 training steps. Verify decreasing loss, correct shifted targets, causal masking, padding, cache save/restore, and cached versus full recomputation. This trial is for implementation problems, not evidence about the research hypothesis.

4. **Train three small transformers normally, then freeze them.** Use ordinary next-token cross-entropy, which penalizes assigning low probability to the observed next token. Select checkpoints using transformer validation only. Freeze all transformer weights before fitting any analysis tools.

5. **Check that each transformer learned the task.** Compare its full next-token and short-future probabilities with the oracle. In particular, verify the noisy cue's delayed effect and the reset's removal of that effect. Do not assume that exact generator equalities imply exact model equalities.

6. **Collect three kinds of model information.** Record the current token's final residual vector, its vectors across all layers, and the specified cache summary. Fit the same small density model and classical baselines separately to each source. Use model probabilities as the principal targets; use oracle beliefs only for a separate information-reading diagnostic.

7. **Test autonomous updates on ordinary histories.** Encode a history once. Then give only identical continuation tokens, position, and the known sequence stage to the compact model. Compare its next distributions with transformer reference distributions. Never replace the predicted compact state with a fresh transformer encoding during the scored rollout.

8. **Apply the same cache-exchange intervention at two locations.** For each matched pair, exchange all earlier positions, just the current position, or both. Keep positions, masks, lengths, and model seed fixed. Append identical continuations. Predict effects from the modified cache summary before revealing the reference results.

9. **Apply the predefined decision criteria.** Report all three seeds, all selected pairs, reset controls, numerical controls, parameter counts, bytes, and runtime. Separate evidence about transformer memory from evidence that the density-matrix constraint helps.

3. One generator, including its own control

3.1 Vocabulary and rules

Use exactly eight tokens: `^`, `0`, `1`, `a`, `b`, `?`, `R`, `$`. Each symbol is one token; do not use a text tokenizer.

An episode has this layout:

```
^  N[1] ... N[16]  C  W[1] ... W[d]  ?  Y  $
```

The hidden bit Z is either 0 or 1 with equal probability. The 16 nuisance tokens N are independent fair draws from `a` and `b`; they reveal nothing about Z. The clue C equals Z with probability 0.8, and is the opposite bit with probability 0.2. Only C is shown to the transformer.

After the clue, repeatedly choose the next token independently of Z:

Next waiting-stage token	Probability	Effect
a	0.4	Keep Z and continue waiting
b	0.4	Keep Z and continue waiting
R	0.1	Replace Z by a new independent fair bit; continue waiting
?	0.1	Leave the waiting stage; next emit the current Z as Y

After Y, emit `$` and end the episode. For fixed-horizon probability comparisons, regard `$` as absorbing: once it is emitted, all remaining positions are `$` by definition. Apply this stopping convention to the transformer and every surrogate as well. Do not use untrained transformer outputs after an end token.

The observer's uncertainty about Z can be summarized by b = P(Z=1 | observed history). This is a posterior probability: a probability updated after seeing evidence. Immediately after clue 0, b=0.2; after clue 1, b=0.8. Fillers and `?` do not change b. An `R` sets b=0.5. Thus one ordinary number plus the observable stage is enough for the true generator.

The nuisance prefix supplies many genuinely different, label-independent histories for splitting and controls. It is not additional hidden structure. The clue always occupies zero-based position 17, after `^` and 16 nuisance tokens. All paired comparisons use the same nuisance prefix, waiting-token string, length, current token, and comparison position.

3.2 Exact worked pair

Take N=(ab)^8, meaning eight repetitions of `ab`. Consider:

```
h0 = ^ (ab)^8 0 a b a b
h1 = ^ (ab)^8 1 a b a b
```

Both histories have 22 tokens and end at zero-based position 21. In the vocabulary order (`^`,`0`,`1`,`a`,`b`,`?`,`R`,`$`), both complete next-token distributions are exactly:

```
(0, 0, 0, 0.4, 0.4, 0.1, 0.1, 0)
```

Their futures differ:

Quantity	After h0	After h1
P(next two tokens are ? 1)	0.02	0.08
P(next two tokens are ? 0)	0.08	0.02
P(next three tokens are a ? 1)	0.008	0.032
P(next token is 1, after appending ?)	0.2	0.8

Total variation distance, or TV, measures probability disagreement: add the absolute probability differences and divide by two. It ranges from 0 for identical distributions to 1 for completely disjoint distributions. The one-token TV here is 0; the two-token joint-future TV is 0.06; the three-token TV is 0.108. After the same `?`, next-token TV is 0.6. These numbers were checked with exact rational arithmetic while preparing this report.

The control pair is h0R and h1R. Both now have a fresh fair Z, the same sequence stage, and the same waiting-token rules. Their distributions over every future sequence are identical, not merely their next-token distributions. Their previously computed cache tensors can still differ. Stored distinctions need not remain behaviorally relevant.

This example uses elementary probability. It is a diagnostic construction, not a mathematical novelty claim.

3.3 Difficulty and shortcut controls

Keep the main experiment at clue reliability 0.8 and query probability 0.1. Difficulty can later be changed by lowering clue reliability or measuring longer delays, but do not introduce a parameter sweep in the first pilot.

The delay is geometric: there is a 0.1 probability of a query on each waiting step. Generate episodes naturally and right-censor at context length 64; right-censoring means stopping observation without inventing an end or query token. Never force a query at the context boundary. Train the transformer on naturally occurring longer delays too. The primary longer-delay test withholds those lengths from the fitted analysis tools, not necessarily from the transformer.

For controlled pairs, sample no-reset waiting strings from `a` and `b`, and evaluate at delays 4, 8, and 12. Test analysis generalization separately at delays 24 and 32. Appending `R` gives the control at a common shifted position. All such strings have positive probability under the same generator. Label forced long no-reset histories as a stress-test distribution and report ordinary generator-weighted results separately.

Balance the two clue values. Independently resample nuisance prefixes across families; keep each pair's nuisance prefix identical. Do not let filenames, split IDs, padding length, sample order, or position encode the clue. All models receive identical position and stage side information. Do not give a surrogate the true posterior, clue identity, reset flag, or future answer as an extra input.

3.4 Data splits without analysis leakage

Hash the 16-token nuisance string with a fixed documented hash and take its value modulo 100. Use buckets 0-69 for transformer training, 70-74 for transformer validation, 75-84 for analysis fitting, 85-89 for analysis validation, and 90-99 for final testing. Hashing affects assignment only; it is never a model input. This grouping makes the evaluated post-clue histories disjoint. Shared very short prefixes before the clue are unavoidable and are not used as analysis examples.

Generate training episodes online with independent random-number streams for each split and seed. Keep the tokenized history family, not individual activation rows, as the split unit. Deduplicate repeated analysis prefixes. Fit normalization, dimensionality reduction, encoders, update rules, diagnostic probes, thresholds chosen on validation, and any hyperparameter decisions only on their designated splits.

Use 20,000 distinct starting histories for analysis fitting and 4,000 for analysis validation, balanced by clue, reset status, and delays 0-12. Collect their ordinary continuation states for fitting update consistency. The final ordinary test uses 1,024 new nuisance families, paired clues, and a prespecified mixture of delays 4, 8, and 12. Use 512 further families for delays 24 and 32. A fixed hash selects 256 ordinary-test pairs and 128 long-delay pairs for complete three-token trees. Keep all final families sealed until settings are fixed.

The group split slightly changes the nuisance-prefix sampling distribution within each split. This cannot change the exact post-clue oracle probabilities because nuisance strings are independent of Z. Scientific scoring starts after the clue, not on the split-conditioned nuisance generator.

4. Implementation-ready settings

4.1 Transformer and hardware

Use a conventional decoder-only causal transformer: every token can attend only to itself and earlier tokens. A pre-normalization block normalizes its input before attention and before its feed-forward network.

Setting	Fixed pilot choice
Vocabulary / maximum context	8 / 64 tokens
Blocks / model width	4 / 128
Attention heads / width per head	4 / 32; ordinary multi-head attention
Feed-forward width / nonlinearity	512 / GELU, a smooth activation function
Position representation	Learned absolute position embeddings, positions 0-63
Normalization / dropout	Pre-LayerNorm, final LayerNorm / 0
Input-output embeddings	Untied; biases enabled; roughly 0.8 million parameters
Initialization	Normal weights, standard deviation 0.02; zero biases; LayerNorm gain 1
Objective	Ordinary shifted next-token cross-entropy; no belief labels
Optimizer	AdamW; learning rate 0.0003; betas (0.9, 0.95); epsilon 1e-8

Setting	Fixed pilot choice
Regularization	Weight decay 0.01 on matrix weights, excluding embeddings, biases, and normalization; gradient norm clipping at 1
Schedule	200 warm-up steps; cosine decay to 0.00003 by step 6,000
Batch / budget	128 independently generated episodes / 6,000 steps per seed
Seeds	17, 29, 43; keep model and data seed mappings in a manifest
Checkpoint selection	Evaluate every 200 steps on 4,096 fixed validation episodes; lowest validation loss
Precision	BF16 training if supported; FP32 analysis with the same explicit attention implementation

The budget is 49.15 million padded token slots per seed; the number of unmasked training targets is smaller and must be logged. Right-pad batches, mask padding both in attention and loss, and include the first end-token prediction in the loss. Do not train on invented targets after termination. Do not pack multiple episodes into a context in this pilot.

Assume one CUDA-capable GPU with 24-32 GB VRAM, at least 32 GB host RAM, and sufficient local disk for checkpoints and compact feature banks. A 24 GB RTX 3090/4090-class card is a reasonable hardware assumption; it is not a measured requirement. This model is small enough that storing all raw per-example caches is more likely to waste resources than model weights. Stream extraction and save only the defined features plus a small audit subset of full caches.

Runtime estimates, not measurements: roughly 0.5-2 GPU-hours per transformer seed and another 2-8 GPU-hours for the complete three-seed analysis, branching evaluations, and baselines, depending strongly on batching and implementation. Reserve about 4-14 GPU-hours for the pilot. Time 100 real training steps and a representative analysis batch before committing to these estimates. No multi-GPU reproduction of [1] is necessary.

The correctness trial uses the same architecture and generator, seed 17, batch 32, and 300 steps. Require functional checks and a clear loss decrease, not the final scientific thresholds. Discard this checkpoint for the formal three-seed runs.

4.2 What each encoder receives

An encoder is a fitted function that compresses selected internal numbers. It is applied to the computational state produced by a particular history. The same fixed weights can produce many different history-specific matrices; a matrix is not extracted from the weights alone.

Let t be the current token position. Record the block-output residual before the next block's normalization; the final-block vector is recorded before the final LayerNorm.

Source	Exact input before common preprocessing	Native size
Final vector	Current token's post-block-4 residual	128 reals
Across layers	Concatenated current-token post-block residuals from blocks 1-4	512 reals

Source	Exact input before common preprocessing	Native size
Cache summary	For each of 4 layers: K and V at the clue, K and V at the current token, and mean K and mean V over positions strictly between clue and current token; concatenate all heads	3,072 reals

For an empty middle interval, use zero vectors. If the current position is the clue, clue/current entries intentionally duplicate. Keys and values are exactly the tensors the inference cache stores, after their projection. The architecture uses absolute position embeddings, so no rotary rephasing is needed.

Standardize each coordinate using analysis-fit means and standard deviations, with a standard-deviation floor of 1e-6. Reduce the second and third sources to 128 coordinates using PCA fitted only on analysis-fit data. PCA means principal component analysis, a linear compression that retains directions with large variance. Keep the first source at 128 coordinates. Do not whiten PCA coordinates. Freeze preprocessing before final testing.

All three then use a 128-to-32-to-3 encoder with GELU between its two linear layers. This is 4,227 fitted parameters. Within each input source, every matched baseline receives exactly the same preprocessed data and the same encoder architecture. Report preprocessing storage and cost separately; comparisons across information sources are not a claim that their native inputs cost the same.

PCA and pooling can discard important information. Therefore, also fit a small ridge-regression diagnostic from each native source to the true posterior b, selecting its penalty on analysis validation. Ridge regression is linear fitting with a penalty on large weights. Report recovery with and without PCA. A failed cache-summary encoder is not evidence that the full cache lacks the information. The selected summary is the precise hypothesis being tested.

4.3 The density state and its updates, in words

Use a 2-by-2 complex density matrix. A density matrix is a Hermitian matrix (equal to its conjugate transpose), positive semidefinite (no negative eigenvalues), with trace one (diagonal entries sum to one). It needs three independent real coordinates. Constrain the encoder's three outputs to the inside of a unit ball and convert them to the matrix using Appendix A. This guarantees validity; it does not guarantee useful predictions.

Fit small token-specific matrices that jointly produce the next-token probabilities and update the density after an observed token. These are Kraus operators: matrices whose normalized products ensure that the state remains valid and that output probabilities add to one. The operators are fitted to the frozen transformer's probabilities, not imported from the known generator.

Use two trainable complex 2-by-2 operators per token, plus a tiny fixed numerical regularization branch. Their raw entries have an intercept and a linear dependence on t/64. Use a separate operator set for each of four observable stages: waiting, answer, end-token pending, and invalid-prefix. A fifth stopped stage is fixed and emits only $. The stage is a deterministic counter updated from supplied tokens, not hidden information. It is supplied equally to every baseline and counted as part of stored state.

The tested compact state is the tuple (density matrix, stage, position), not the matrix alone. This is a small, explicitly stage- and position-conditioned quantum-instrument surrogate, not a claim that the transformer is a quantum system or that its whole computation is a stationary two-dimensional

HQMM. An HQMM, or hidden quantum Markov model, is a sequence model whose hidden state is a density matrix and whose symbol operations generate probabilities and conditional updates. [7-8]

There are 1,024 real fitted operator parameters: 4 stages x 2 time coefficients x 8 tokens x 2 operators x 8 real entries. Together with the encoder, the matched density model has 5,251 fitted parameters, excluding common preprocessing. No extra readout network is fitted; the operators themselves supply the probabilities.

Fit the encoder and operators together after freezing the transformer. Use Adam, learning rate 0.001, betas (0.9,0.999), no weight decay, clipping at 1, batch 256, and 3,000 updates. Use three initialization seeds per transformer/source/representation; choose using analysis validation and report restart variability. This is a small fit, not another transformer training run.

The loss matches full eight-token teacher distributions at rollout offsets 0-4, plus 0.1 times squared update-consistency distance to a freshly encoded ordinary continuation state. A teacher is the frozen transformer being approximated. The fresh encoding is used only as a training target and diagnostic, never as a replacement rollout state. Use equal weight per rollout offset and per clue/reset stratum.

Half the fitting continuations are sampled from the generator; half are supported strings with a query placed uniformly at one of the first four positions, with fillers drawn fairly and the subsequent bit drawn from the oracle conditional distribution. These are observations supplied to the model, not internal interventions. Collect their teacher probabilities using the full normal prefix. Fit no parameters on patched caches or final-test histories.

For the density model, consistency distance is trace distance, defined in Appendix A. Use total variation between state probabilities for the classical HMM and Euclidean distance divided by 2*sqrt(3) for the bounded three-number neural state. Squaring these distances gives a comparable bounded consistency penalty. These geometries remain different modeling choices; report the penalty and do not treat it as evidence of quantum mechanics.

4.4 Classical comparisons that must be included

Representation	Persistent numeric memory	Fitted dynamics/readout parameters	Purpose
Density matrix, dimension 2	3 independent reals	1,024	Candidate constraint
Classical HMM with 4 latent states	4 probabilities, store 3 and infer the last	1,024	Equal-dimensional probability-state comparison
Bounded recurrent neural state	3 reals	1,010	Flexible classical update comparison
Classical HMM with 2 latent states	1 independent probability	256	Lean diagonal-density diagnostic
Exact oracle posterior b	1 probability, supplied from known rules	No fitted dynamics	Task-level reference, not an activation-based competitor

The four-state HMM uses positive token-labeled 4-by-4 transition matrices, normalized across emitted token and destination for each source state. Raw entries have the same four stage choices

and linear t/64 dependence. Encode initial probabilities as softmax(u1,u2,u3,0), so the encoder still has three outputs. Appendix A gives the updates.

For the three-number recurrent baseline, use tanh-bounded encoder and update outputs. The update network has inputs state(3)+token(8)+stage(4)+time(1), a hidden layer of width 27 with GELU, and 3 outputs. The probability network has inputs state(3)+stage(4)+time(1), width 27, and 8 softmax outputs. These have 543 and 467 parameters respectively. A softmax converts arbitrary scores into nonnegative probabilities summing to one. This baseline is not described as a formally minimal linear predictive-state representation.

Each three-number model stores 12 bytes in FP32, plus two integers for stage and position under the same convention. Reconstruct the matrix or fourth probability when needed; also disclose actual working-memory allocation. A literal complex matrix may occupy more bytes. All matched models pay the same source-extraction and PCA costs. The two-state model uses a one-output encoder and has fewer parameters; it is a simpler diagnostic, not the parameter-matched opponent.

Identical parameter counts do not imply identical computation. Report training time, peak VRAM, and prediction latency at batches 1 and 256. Give classical baselines at least 3,000 updates and then, if faster, additional updates up to the density fit's measured training-time budget, selecting by validation only. Use the same restart budget. A density claim must survive this matched-time comparison. Do not count an HMM with two states as the only classical competitor to a matrix with three real degrees of freedom.

5. Measurements, interventions, and predefined decisions

5.1 Comparing histories properly

Let P_G be the exact generator distribution, Q_T the trained transformer's distribution, and Q_D the compact surrogate's distribution. Keep these names separate in code and figures. Compare the entire eight-token distributions, including probability incorrectly placed on impossible tokens.

The same sampled token only means two random draws happened to agree. The same most likely token only means their largest-probability entries agree. Neither implies equality of the full distributions. In this experiment, pair matching uses TV between complete Q_T distributions, not sampled tokens or argmax labels.

First generate candidate pairs by the oracle rules. For the model-matched subset require TV(Q_T(.|h0), Q_T(.|h1)) <= 0.01. Freeze this tolerance before inspecting intervention results. Report the fraction retained and report results for all oracle-matched pairs as well; a tiny selected subset would be misleading. At least 90% of the ordinary pairs should qualify for a clean pilot.

For horizons 1, 2, and 3, enumerate all 8^k token strings, using the absorbing-end convention. Multiply conditional probabilities along each branch to obtain the joint distribution. Do not substitute separate token marginals for the joint. Include invalid-prefix branches from the model's own probabilities; do not renormalize away their mass. The surrogate's invalid-prefix stage makes its recurrence defined on these branches too, even though their behavior is poorly constrained by natural data.

At longer horizons, avoid an exponentially large tree. Test distributions along fixed, shared supported continuations and report probabilities of a prespecified small set of strings. Such a set is not a complete joint-future distribution. Use continuations `?`, `a b ?`, and `(a b)^3 ?` for the causal endpoints, plus randomly sampled supported eight-token continuations for update testing.

Measure trace distance between density matrices and compare it with actual TV differences of Q_T futures, within matched position and stage. Report correlation and calibrated error, not a pretty scatter alone. Include the reset pairs: large matrix distances with identical behavior indicate that the matrix retains distinctions irrelevant to that prediction task. Matrix coordinates are not uniquely identified semantic variables.

5.2 The cache exchange

For a recipient history h0 and donor h1 with the same length t+1, divide each layer's cache into E, all positions strictly before t, and C, position t itself. Exchange keys and values together, across all heads and all layers, without changing their indices. Make four copies:

Cache used for the continuation	Earlier positions E	Current position C
00: recipient baseline	Recipient	Recipient
10: earlier-memory exchange	Donor	Recipient
01: current-memory exchange	Recipient	Donor
11: complete donor cache	Donor	Donor

This is one intervention type with two locations. The complete donor exchange is a correctness check, not a causal discovery. The partial exchanges ask whether either part can change the outcome while the other is held fixed. Their effects can interact; do not assume they add up or interpret small separate effects as absence of redundant information.

Use donor tensors from real, equal-length histories processed by the same model. Match the nuisance prefix, waiting suffix, last token, phase, absolute positions, attention mask, and precision. Preserve padding masks, cache lengths, and next-position IDs. Do not average arbitrary activation vectors or apply an inverse probe to invent the primary intervention. A hybrid cache can still be inconsistent with every natural history; interpret it as a surgical computation intervention, not as a naturally occurring belief. Track probability mass on impossible tokens to detect gross disruption. [12]

After editing the cache, append a prescribed identical token before scoring new transformer predictions. Cache edits do not retroactively change logits already computed for the recipient. In particular, append `?` and then score the following answer distribution, or append `a b ?` and score after the final `?`. Do not recompute the original prefix, because that would overwrite the intervention.

For the density/cache encoder, encode the modified cache once, before feeding the continuation. Predict all later distributions by updating that state. For the residual-only encoders, the saved current residuals remain the recipient's numbers after this cache-only intervention. Record this explicitly: their failure to anticipate a cache change that they cannot observe is a state-access limitation, not a fair density-versus-classical contest. The fair representation comparison under intervention is within the cache-input condition.

For every arm, record the change in P_T(1) after the common query, the full distribution's TV change, and the change in the log probability ratio of 1 to 0. The log ratio checks that probability saturation is not hiding effects. Record the surrogate's predicted change before inspecting its error against the transformer. Evaluate both transplant directions.

Run two controls. First, swapping a cache with itself must have no measurable effect. Second, repeat the partial exchanges on matched h0R/h1R controls, optionally followed by identical fillers. The

oracle futures are equal, so a well-trained model should show little systematic cue transfer. Large reset-control effects require investigation before interpreting main effects. Require mean impossible-token mass after the query to stay at most 0.01 in each partial arm; otherwise flag that arm as disrupted rather than cleanly localized.

Successful swaps implicate a cache block carrying the clue's effect. They do not prove that individual density-matrix entries are independently manipulable causal variables. Accurate density predictions support this summary for these interventions, without establishing a unique internal mechanism.

What this says about location. The final residual vector is computed during a forward pass; it is not normally carried forward as an independent recurrent state. The persistent information available to the next token is in the KV cache and position bookkeeping. A current residual can reveal information also stored elsewhere. Patching the final post-block vector after all caches have been written, while forcing the same next token, should not affect subsequent predictions in this architecture; that is an architectural negative check, not a new memory result. Current-position KV and earlier-position KV are the relevant causal storage comparison.

5.3 Predefined gates

These are practical pilot tolerances, not universal scientific constants. They are chosen relative to the oracle's 0.60 answer-probability gap, 0.06 two-token TV gap, and expected numerical noise. Use paired bootstrap intervals over nuisance families, resampling whole families with all their positions and arms. Report each transformer seed separately; three seeds do not justify strong population-level significance claims.

Gate	Predefined requirement	If it fails
Generator	Exact normalization and worked-pair identities; numerical oracle error <1e-12; prove reset equality for all futures and enumerate short trees as a check	Fix the generator/oracle before training
Cache implementation	On 1,024 prefixes, FP32 cached and full-recomputed logits differ by <=1e-5 and full-distribution TV by <=1e-6; self-swap and full donor replay meet the same TV tolerance	Fix positions, masks, precision, or cache updates; no scientific interpretation
Transformer learning	Waiting-stage mean TV to oracle <=0.01; query P(1) mean absolute error <=0.03 and 95th percentile <=0.08; impossible-token mass mean <=0.005	Increase training or simplify only in a new declared run; do not evaluate the state hypothesis yet
Pair validity	>=90% meet current-model TV <=0.01; mean query answer gap >=0.50; report all pairs too	The required matched-now/different-later behavior is not cleanly realized
Reset learning	Mean full next-distribution distance between reset-pair query outputs <=0.03	The model has not learned the intended erasure control
Ordinary state prediction	Mean TV to Q_T <=0.02 at roots; complete three-token joint TV <=0.05	The selected compact model is not a useful ordinary behavior summary
Density updates	Mean one-step trace distance to fresh clean encoding <=0.05; <=0.10 after 8 supplied tokens; output TV <=0.03 at 4 steps and <=0.05 at 8	Revise this encoder/update pair; if outputs pass but trace distance fails, claim only predictive agreement

Gate	Predefined requirement	If it fails
Held-out longer delays	Transformer query error <=0.05 at delays 24 and 32; surrogate output TV <=0.05 and joint-three-token TV <=0.07	Separate transformer failure from analysis generalization failure; do not blame one for the other
Causal memory effect	At least one partial arm has donor-directed median P(1) change >=0.20 and its family-bootstrap 95% lower bound >0.10; reset partial-arm mean absolute effect <=0.03	No clean causal localization under this intervention; whole-cache success alone does not pass
Predicting intervention effects	Mean absolute error of the predicted signed P(1) change <=0.05; direction correct in >=90% of cases with actual absolute effect >=0.10; conditional future-distribution TV <=0.05	Observational prediction did not transfer to the selected interventions
Density-specific value	Pass absolute gates and beat the best matched classical model by >=20% relative and >=0.01 absolute on both joint-three-token TV and intervention-effect error; paired interval excludes zero within each seed; retain benefit in time-matched fits	Do not recommend density matrices as the preferred description
Seeds	All three transformer seeds meet correctness and learning gates; effect direction and the relevant success criteria replicate in all three	Two of three is tentative and merits repetition; one of three is an unstable pilot

Do not require a correlation threshold for geometry as the main success criterion. Predictive and causal errors matter more. Report seed ranges, quantiles, and family-level intervals alongside means. Prespecify all endpoints and both partial arms; do not search many layers and report only the strongest result.

5.4 How to interpret outcomes

Useful memory finding, density not helpful: The transformer passes learning checks; partial swaps causally alter delayed predictions; the cache-based classical state predicts those effects as well as or better than the density model. Continue the memory investigation with the simpler model. This is not a failed project.

Evidence that the density constraint helps here: The density model passes ordinary, update, delay, and intervention tests and beats both capacity-matched and time-matched classical fits consistently. This supports this inductive bias, meaning this restriction on the kinds of models that can be learned, for this task and budget. It does not demonstrate quantum computation or an irreducible quantum memory advantage.

Revise: The transformer is inaccurate, cache parity fails, reset interventions damage behavior, or all compact models fail even on ordinary histories. Those failures do not test the intended scientific question cleanly. If the native-cache posterior probe succeeds but its PCA version fails, revise preprocessing rather than declaring memory absent.

Evidence against this particular approach: The density model is valid but predicts poorly, cannot update without repeated teacher access, works only on fitting histories, loses to matched classical models, or mispredicts genuine held-out cache interventions despite successful ordinary prediction. A representation may be decodable yet not a sufficient causal summary. Do not fit intervention outcomes after inspecting final tests and call the resulting performance held out.

A positive pilot would justify a larger, freshly specified study and another novelty search. It would not by itself establish publication readiness, uniqueness of the representation, or applicability to language models generally.

6. Compact overlap table

Primary source	Already establishes or investigates	Boundary relative to this pilot
[1] Riechers, Elliott & Shai, 2025, arXiv:2507.07432	Activation recovery of classical, quantum, and post-quantum predictive states; future information is already present	This pilot tests a learned surrogate of Q_T under partial cache exchanges
[2] Shai et al., NeurIPS 2024	Known belief geometry; RRXOR next-token ambiguity; across-layer recovery	Neither the ambiguity nor across-layer probing is new
[3] Piotrowski et al., ICML 2025	Mechanistic account of constrained belief updates in small transformers	Update mechanisms and geometric predictions are existing contributions
[4] Balcells et al., September 2026	In-context belief probes; next-token-preserving belief-direction steering; prediction interventions	Closest intervention overlap; this pilot adds a narrowly specified autonomous cache-transfer comparison
[5] Balcells & García Segura, public capstone, 2026	Decoded-belief injections and next-token effects	Broad causal belief injection is already implemented
[6] Littman, Sutton & Singh, 2001	States defined through future tests and recursive updates	Predictive-state mathematics is established
[7-8,16] HQMM foundations and learning papers, 2010-2020	Density-state probabilities, conditional updates, learning, links to observable-operator models	Matrix validity and HQMM fitting are established methods
[9] Li et al., ICLR 2023	Synthetic-task world-state probing and causal editing	Internal world-state interventions are established
[10-11] Cache Steering, 2025; Memory Inception, 2026	Causal control through saved attention information	Editing KV memory itself is not novel
[12] Zhang & Nanda, ICLR 2024	Patching results depend on interventions and metrics	Motivates real matched donors, full probabilities, and log-ratio controls
[13] J-state closure, public research repository	Compact states, persistent-channel tests, and intervention-retention protocols	Broad compact causal-state claims overlap an existing research program
[14] Ning, August 2026 preprint	New constrained HQMM learning method and benchmark comparisons	Does not test causal summaries of transformer caches

The table is an overlap assessment, not a ranking of source reliability. Published papers, preprints, and public project code are explicitly distinguished.

Appendix A. Optional mathematics

A1. Probability definitions and the oracle

For any distribution p and q over a finite set X,

```
TV(p,q) = (1/2) sum_{x in X} |p(x)-q(x)|.
KL(p||q) = sum_{x:p(x)>0} p(x) log[p(x)/q(x)].
```

KL is Kullback-Leibler divergence, an asymmetric discrepancy measured here using natural logarithms. It is supplementary to TV. Generator-zero terms contribute zero; do not renormalize Q_T to the allowed vocabulary subset.

With a fair hidden Z and clue reliability r=0.8, Bayes' rule gives P(Z=1|C=1)=r and P(Z=1|C=0)=1-r. All waiting emissions are independent of Z. A reset resamples Z independently, proving equality of all future distributions from h0R and h1R. This is a standard conditioning argument.

For a model Q and a continuation w=(x1,...,xk),

```
Q(w|h) = product_{j=1..k} Q(xj | h,x1,...,x[j-1]).
```

This chain rule is not a new result. In the worked example, differences in two-token futures occur only at `? 0` and `? 1`, each with absolute difference 0.06, yielding TV=0.06. At horizon three, a query can occur at the first step (contribution 0.06) or after a non-reset filler (contribution 0.8*0.06=0.048), yielding 0.108.

For the generator, a sufficient waiting-stage state is (b,stage), where b is the posterior. It is not necessarily sufficient for the learned transformer's small history-dependent errors. That distinction motivates using Q_T, not P_G, as the surrogate target.

A2. Constructing the two-dimensional density

The fitted encoder gives u in R^3. Let r = u / sqrt(1 + u.u). Then ||r|| < 1. Construct

```
rho(r) = (1/2) [[1+r_z,   r_x - i*r_y],
                [r_x+i*r_y, 1-r_z     ]].
```

Here i is the imaginary unit. The eigenvalues are (1+||r||)/2 and (1-||r||)/2, so the matrix is positive and has trace one. These three real coordinates are often called a Bloch vector; this name does not imply that the neural model uses a physical qubit.

The exact generator can be represented with rho_G=diag(1-b,b). Its off-diagonal entries are zero in one fixed basis. Therefore the present task does not require noncommuting quantum states. Noncommuting means that exchanging the order of matrix multiplication changes the result. Off-diagonal entries in a fitted basis alone do not establish nonclassical information; a common basis change can alter them.

For two such matrices, trace distance is

```
D_tr(rho,sigma) = (1/2) ||rho-sigma||_1
                = (1/2) ||r(rho)-r(sigma)||_2,
```

where the first norm is the sum of singular values and the second is ordinary Euclidean length. The oracle pair has trace distance 0.6 despite identical next-token distributions; after reset it is zero. These facts describe this representation, not the trained model's unknown state.

A3. Valid learned probabilities and updates

For stage s and position t, form two raw complex 2-by-2 matrices per token:

```
B[s,x,j](t) = B0[s,x,j] + (t/64)*B1[s,x,j], j=1,2.
B[s,x,3](t) = sqrt(epsilon/8) * I, epsilon=1e-6.
G = sum_{x,j} B[s,x,j](t)^dagger B[s,x,j](t).
K[s,x,j](t) = B[s,x,j](t) G^(-1/2).
```

Dagger means conjugate transpose. The fixed third branch makes G positive definite. Compute its inverse square root with a stable 2-by-2 Hermitian eigendecomposition; do not clip an invalid matrix and silently claim exact normalization. Because all branches are included,

```
sum_{x,j} K[s,x,j]^dagger K[s,x,j] = I.
A_x(rho) = sum_j K[s,x,j] rho K[s,x,j]^dagger.
q_D(x|rho,s,t) = trace[A_x(rho)].
U_x(rho,s,t) = A_x(rho) / q_D(x|rho,s,t).
```

The probabilities are nonnegative and sum to one, and the conditional updated state is valid. The readout can equivalently be written trace(E_x rho), where E_x=sum_j K_xj^dagger K_xj. Each E_x is positive and the E_x sum to I. These are standard quantum-instrument equations. [7-8,16]

For differentiable implementation, a useful 2-by-2 formula avoids differentiating eigenvectors near repeated eigenvalues. Let a=sqrt(det(G)) and c=sqrt(trace(G)+2a). Then G^(-1/2)=c*(G+a*I)^(-1). Compute real traces and determinants after checking Hermitian symmetry; use double precision for these tiny normalizations if needed. This is a standard matrix identity, not a proposed contribution.

The stage update is deterministic: waiting plus `?` becomes answer; waiting plus `a,b,R` remains waiting; answer plus `0` or `1` becomes end-pending; end-pending plus `$` becomes stopped. Every other non-end transition becomes invalid. In the invalid stage, `$` stops and all other tokens remain invalid. For the stopped stage, emit `$` with probability one. The position increments by one until the context boundary. This specifies all branches without leaking the cue into the side information.

The generator itself admits a diagonal realization. In the waiting stage use sqrt(0.4)I for each filler, sqrt(0.1)I for `?`, and two reset branches sqrt(0.05)I and sqrt(0.05)X, where X exchanges basis states 0 and 1. On diagonal densities the reset output is I/2 after normalization. In the answer stage use the two basis projectors for tokens 0 and 1; in end-pending emit `$`. This construction is only a correctness reference; do not use its known parameters to fit the transformer surrogate.

A4. Classical alternatives and costs

For the four-state classical model, let b be a four-entry probability column vector. For each stage and position, normalize raw scores over (token x, destination j) separately for each source i:

```
M_x[j,i] = softmax_{x,j}(A0[s,x,j,i] + (t/64)*A1[s,x,j,i]).
q_C(x|b,s,t) = 1^T M_x b.
b_next = M_x b / q_C(x|b,s,t).
```

All entries are nonnegative and sum_{x,j}M_x[j,i]=1. Therefore the probabilities and conditional updates are valid. There are 4*2*8*4*4=1,024 raw parameters. The density model's 1,024 raw parameters include complex real/imaginary entries; comparing just Hilbert dimension 2 against four classical states would obscure their equal three-real-number state storage.

The density computations involve complex products and a small inverse square root, so they may cost more despite equal parameter counts. Report this directly. No conclusion here concerns physical quantum memory, quantum speedup, or infinite-precision computational advantages.

A5. Fitting and evaluating state closure

For one root h, start with z0=D(F(S_h)), where F is the fixed source extraction/preprocessing. Recursively update z[k+1]=U_{x[k+1]}(z[k],s[k],t[k]) using only supplied tokens and deterministic side information. Train with

```
L = mean_{k=0..4} KL(Q_T(.|h,x[1:k]) || q(.|z[k],s[k],t[k]))
    + 0.1 * mean_{k=1..4} d(z[k], stop_gradient(D(F(S_hx[1:k]))))^2.
```

Use balanced sampling as specified in Section 4. Targets and clean encodings may be precomputed; the transformer remains frozen. The stop-gradient operation prevents the consistency term from moving its current target in the same optimization step. Each target encoding also receives ordinary supervised losses when its prefix appears as an example. Stop at episode termination rather than treating padding as data.

At evaluation there are no teacher refreshes. Separately report one-step updates initialized from a fresh state and 4-/8-step updates initialized only once. The latter test is harder. Fitting a decoder at every freshly observed transformer state does not establish it.

For ordinary states at a common position and stage, a valid instrument gives an important check: the TV distance between its complete k-token future distributions cannot exceed the initial trace distance. A complete future string is one outcome of a combined measurement. This is an established distinguishability bound, not a proposed discovery. It does not apply in the same form to distributions conditioned on a rare forced continuation; conditioning can amplify differences.

When comparing true model futures to surrogate geometry, the triangle inequality adds approximation error:

```
TV(Q_T^k(h),Q_T^k(h'))
 <= TV(Q_T^k(h),Q_D^k(h)) + D_tr(rho_h,rho_h')
    + TV(Q_D^k(h'),Q_T^k(h')).
```

Thus matrix separation alone is insufficient; the two prediction-error terms must be measured. Short-horizon agreement also does not prove equality of all future distributions.

A6. Causal endpoint and its scope

For a fixed continuation c ending in `?`, define the effect of partial swap a as

```
Delta_T(a,c) = Q_T(1 | edited_cache_a, supplied c)
               - Q_T(1 | recipient_cache, supplied c).
```

Predict Delta_D from the two initial cache encodings and their autonomous updates. Compare signed error, absolute error, and direction, alongside full-distribution effects. A hybrid cache does not have a generally defined generator posterior, so do not score it against an invented P_G target.

For interaction, report f11-f10-f01+f00, where f is the answer probability in each cache arm. This is a standard two-factor contrast. It reveals non-additivity; it is not a percentage of information located in either block.

The intervention changes persistent variables after the current logits exist. Hence the tested estimand is the distribution after one or more externally supplied tokens. This avoids attributing a change to an output that was already computed before intervention.

Appendix B. Concise pseudocode

B1. Generator and exact oracle

```
V = ['^', '0', '1', 'a', 'b', '?', 'R', '$']

def episode(rng, nuisance):
    z = rng.fair_bit()
    cue = z if rng.uniform() < 0.8 else 1-z
    out = ['^'] + nuisance + [str(cue)]
    while len(out) < 64:
        x = rng.choice(['a','b','?','R'], p=[.4,.4,.1,.1])
        out.append(x)
        if x == 'R':
            z = rng.fair_bit()
        elif x == '?':
            return (out + [str(z), '$'])[:64]
    return out  # censored, not an artificial end

def oracle_waiting_distribution():
    return dict(zip(V, [0,0,0,.4,.4,.1,.1,0]))

def oracle_answer_probability(cue, has_reset):
    return .5 if has_reset else (.8 if cue == 1 else .2)
```

The hidden reset bit is sampled for generating Y. The oracle probability remains 0.5 after reset because that bit has not been observed. For full-tree code, carry posterior b and stage rather than reading the simulator's hidden sampled bit.

B2. Transformer fitting and analysis fitting

```
families = split_by_hash_of_nuisance_prefix()
for seed in [17, 29, 43]:
    T = SmallCausalTransformer(config, seed)
    for step in range(6000):
        tokens, padding = sample_training_batch(families.train)
        # Inputs and targets are shifted; mask pads and terminal tail.
        loss = next_token_cross_entropy(T, tokens, padding)
        optimizer_step(loss)
        validate_and_checkpoint_on_schedule(T)
    T = select_by_transformer_validation().float().eval()
    freeze(T)
    assert_cache_replay_matches_full_forward(T)

    fit_bank = collect_clean_prefix_and_continuation_states(T)
    for source in ['last_residual', 'all_layers', 'cache_summary']:
        preprocess = fit_standardization_and_PCA(fit_bank, source)
        for family in ['density2', 'classical4', 'vector3', 'classical2']:
            D, U, readout = initialize_surrogate(family)
            for batch in analysis_fit_batches:
                z = D(preprocess(batch.root_features))
                loss = 0
                for k in range(5):
                    loss += KL(batch.teacher_probs[k], readout(z))
                    if k < 4 and not batch.stopped[k]:
                        z = U(z, batch.token[k], batch.stage[k], batch.t[k])
                        loss += .1 * consistency_to_clean_target(z, k+1)
                update_only_surrogate(loss)
            select_checkpoint_on_analysis_validation()
            freeze(D, U, readout, preprocess)
```

This is algorithmic pseudocode, not a complete library implementation. Batch masks handle episodes ending before four steps. Explicitly pass stage and position to each readout as well as each update.

B3. Full joint distributions and intervention evaluation

```
def joint_tree(state, k, readout, update):
    if k == 0:
        return {(): 1.0}
    if state.stopped:
        return {tuple(['$']*k): 1.0}
    p = readout(state)             # every vocabulary token
    result = {}
    for x in V:
        child = update(copy_state(state), x)
        for tail, mass in joint_tree(child, k-1, readout, update).items():
            result[(x,)+tail] = p[x]*mass
    return result

for pair in sealed_test_pairs:
    S0, S1 = prefill_both_histories(T, pair)
    record_full_next_distributions_and_matching_status(S0, S1)
    t = len(pair.h0)-1
    for earlier_from_donor, current_from_donor in [(0,0),(1,0),(0,1),(1,1)]:
        C = clone_cache(S0.cache)
        if earlier_from_donor:
            C[:, :, :t, ...] = S1.cache[:, :, :t, ...]
        if current_from_donor:
            C[:, :, t:t+1, ...] = S1.cache[:, :, t:t+1, ...]
        # Use your implementation's actual [layer,K/V,position,...] layout.
        z0 = D(preprocess(cache_features(C, t)))
        for continuation in ['?', 'ab?', 'abababab?']:
            z = clone(z0)
            reference_cache = clone_cache(C)
            stage, pos = WAIT, t
            for token in list(continuation):
                z = U(z, token, stage, pos)
                stage = next_stage(stage, token)
                pos += 1
                reference = T.step(token, reference_cache, position=pos)
            save_prediction_before_scoring(readout(z, stage, pos))
            compare_to_full_reference_distribution(reference)
```

Do not use the previously saved recipient logits as the post-intervention reference. Copy caches deeply enough that branching does not mutate another arm. Use the same procedure for reset and self-swap controls.

Appendix C. Search scope, source access, and code audit

C1. Search scope

Search date: 21 October 2026, UTC. Searches combined the three supplied titles with terms including predictive state, belief geometry, next-token degeneracy, interventions, KV cache, hidden quantum Markov models, density matrix, causal memory, and 2025/2026 follow-up work. Both general web-search systems were used, with recent-date searches for follow-up papers. Primary arXiv manuscripts, conference proceedings, author-hosted PDFs, and public code were preferred over aggregators.

This was a targeted literature investigation, not an exhaustive citation-database review. Search ranking, indexing delays, inaccessible submissions, and uninspected repositories limit a novelty conclusion. Recheck the closest 2026 papers before registering or submitting a thesis contribution.

C2. What was inspected

Source	Inspection depth and relevant implementation evidence
[1]	Main representation and result sections; conditional future-state definitions; training and regression Appendices E-F; code Appendix H; quantum-public README, run_regression_analysis.py, and regression.py. The executed pipeline entry point combines layer activations, deduplicates prefixes, and invokes regression analysis. The repository also contains alternate regression helpers; code existence does not establish that every helper produced a paper figure.
[2]	Methods and results, especially RRXOR and layer comparison; Appendix A training, regression, and reproduction instructions. Shared epsilon-transformers code was inspected. The original Figure6/Figure7 notebooks were named in the paper but not independently recovered and executed.
[3]	Published proceedings PDF, methods, mechanistic results, Appendices B-G; intermediate_representations.ipynb source cells. The notebook reads residual activations, embeddings, OV terms, and attention patterns and checks a decomposition numerically.
[4]	Full PDF downloaded after the browser retrieval limit; Sections 5-8, relevant appendix discussion and comparisons; public README; run_belief_steering.py and run_prediction_interventions.py. Code includes next-token-matched donor directions, copied prefix caches, residual hooks over a tail, and full-vocabulary probability normalization.
[5]	README and interventions/steering.py; inspected target-belief choices, activation edits, and KL-to-original/injected targets. No project results reproduced.
[6-8,16]	Predictive-state definitions and updates; HQMM construction and learning methods/results; Adhikary repository README describing its MATLAB/Python optimization setup.
[9-12]	Relevant intervention methods and limitations; cache-position alignment appendix in [10]; Othello intervention description; patching method/metric comparisons in [12].
[13-15]	Public closure-project protocol/status documentation; recent HQMM algorithm/results/proof appendix; numerical cache-equivalence paper. These are additional leads and cautions, not replications.

C3. Access limitations

The OpenReview result titled **Understanding the Predictive-State Bottlenecks of Transformer Models**, identifier `58JxyfvZJj`, was surfaced by search but its PDF returned a browser challenge/HTTP 403. Its detailed methods, results, authorship, and overlap were not verified. Do not rely on its search snippet as an established result. This remains a novelty-check lead. [17]

Some arXiv HTML and GitHub page requests failed through the browsing service. Most material needed here was subsequently obtained through public PDFs or public raw code. In particular, the 16.6 MB PDF for [4] exceeded the browser's retrieval limit but was downloaded and read locally; it is not an unread abstract-only source. The proceedings PDF for [3] was also obtained successfully through its public raw link.

GitHub repository-tree API requests returned HTTP 403. Known public raw code files remained readable. The original [2] Figure6/Figure7 notebooks were not individually audited. No claim is made to have inspected every file in any repository.

The [1] regression entry point uses a private company S3 loader. That private bucket was not accessed. Its README advertises public checkpoint/analysis mirrors on Hugging Face, but those checkpoints and training logs were not downloaded or rerun for this investigation. The [4] repository says its results directory is empty and data can be requested; unpublished result files were not obtained.

No separate runnable code link for the August 2026 HQMM preprint [14] was found in the inspected article. Its benchmark claims are therefore reported as the paper's claims, not independently validated comparisons. The proposed pilot does not require reproducing its new optimizer.

C4. Evidence retained and not claimed

The exact worked-pair computations yielded TV values 0, 0.06, and 0.108 for horizons 1-3. A random test of the proposed normalized operator construction had completeness error about 3.3e-16 in double precision. These are small algebraic implementation checks only. All training quality, intervention effects, density performance, seed consistency, VRAM usage, and runtimes in this report remain unmeasured.

References and primary links

[1] Riechers, P. M., Elliott, T. J., and Shai, A. S. Neural networks leverage nominally quantum and post-quantum representations. arXiv:2507.07432, 2025; retrieved full text includes v2. Paper. Public code, quantum-public branch.

[2] Shai, A. S., Marzen, S. E., Teixeira, L., Gietelink Oldenziel, A., and Riechers, P. M. Transformers represent belief state geometry in their residual stream. NeurIPS 2024; arXiv v3 inspected. Paper. Shared code.

[3] Piotrowski, M., Riechers, P. M., Filan, D., and Shai, A. Constrained Belief Updates Explain Geometric Structures in Transformer Representations. ICML 2025, PMLR 267:49399-49419. Proceedings. Analysis notebook.

[4] Balcells, D., Lee, A. J., Rastogi, C., Riechers, P. M., Shai, A., and Poncini, X. Large Language Models Develop Belief State Geometry In-Context. arXiv:2609.17376, September 2026 preprint. Paper. Code.

[5] Balcells, D., with García Segura, M. E. Probing Belief States in Transformer Models. Public ARENA capstone repository, 2026. Project and code.

[6] Littman, M. L., Sutton, R. S., and Singh, S. Predictive Representations of State. NeurIPS 2001. Primary PDF.

[7] Srinivasan, S., Gordon, G., and Boots, B. Learning Hidden Quantum Markov Models. AISTATS 2018, PMLR 84. Proceedings and PDF.

[8] Adhikary, S., Srinivasan, S., Gordon, G., and Boots, B. Expressiveness and Learning of Hidden Quantum Markov Models. arXiv:1912.02098; AISTATS 2020. Paper. Code.

[9] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. ICLR 2023. Paper. Code.

[10] Belitsky, M., et al. KV Cache Steering for Controlling Frozen LLMs. arXiv:2507.08799, 2025; v2 inspected. Paper. Code.

[11] Liu, A. Z., Zhang, M., Greenberg, I., Alnasser, A., Baker, L., and Sous, J. Memory Inception: Latent-Space KV Cache Manipulation for Steering LLMs. arXiv:2605.06225, 2026 preprint. Primary PDF.

[12] Zhang, F., and Nanda, N. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. ICLR 2024. Primary PDF.

[13] skchen17. J-state closure: causal tests of Jacobian-lens J-state closure and compact cognitive controllers. Public research repository, accessed 21 October 2026. Project and protocol status.

[14] Ning, N. Newton-Schulz Retraction-Based Inference Enables Hidden Quantum Markov Models to Outperform Classical HMMs. arXiv:2608.06554, August 2026 preprint. Full text.

[15] Chodavarapu, R., and Xu, L. The Illusion of Equivalence: Systematic FP16 Divergence in KV-Cached Autoregressive Inference. arXiv:2604.15409, 2026 preprint. Primary PDF. Used as a numerical-check lead; its broad causal conclusions are not assumed here.

[16] Monras, A., Beige, A., and Wiesner, K. Hidden Quantum Markov Models and non-adaptive read-out of many-body states. arXiv:1002.2337; version 2, 2012. Primary PDF.

[17] Understanding the Predictive-State Bottlenecks of Transformer Models. OpenReview search lead; full text inaccessible in this investigation. Unverified submission link.